Evidence-Based Candidate Evaluation Agent

> **Frontier Engineering Challenge 2026 (Micro1) — Agentic Workflows Hackathon** Submission. This branch adds an **agentic candidate-evaluation pipeline** to ResumePilot AI and measures, with a reproducible evaluation harness, whether it beats the pre-existing single-pass baseline.

One-line pitch

A multi-agent pipeline (Requirement Extraction → Evidence Extraction → Matching → Verification) that evaluates a candidate against a job description by grounding every verdict in a **quoted line of the resume** — instead of emitting one unverifiable holistic score.

Who has the problem (the intended user)

Recruiters and hiring engineers screening candidates against written job descriptions. They are drowning in resumes and want a fast, *trustworthy* first pass — one they can audit. A score is only useful if the person making the hiring decision can see *why* it was given.

What is the bottleneck (the problem)

The pre-existing ResumePilot workflow (and most ATS tools) produce a **single-pass holistic judgement**: resume + JD → "Candidate score: 82%". The failure mode is **silent over-confidence**:

- a skill listed in a skills section with **no usage context** is scored as *supported*;
- "cloud experience" is treated as **AWS**;
- an absent technology is **hallucinated into SUPPORTED**;
- the headline number hides all of it — there are no quotes, no per-requirement audit trail, nothing a human can verify.

A single opaque score cannot be audited, and an audited-but-wrong score is worse than no score.

How the agent solves it (the value)

Instead of one holistic read, the pipeline decomposes the task into specialised agents, each with a narrow job:

1. **RequirementExtractor** — turns the JD into a concrete, typed list of requirements (splitting compound items like "Python and FastAPI").
2. **EvidenceExtractor** — reads the resume and produces structured `{skill, exact quote, source section, confidence}` claims. It never scores — it only extracts and quotes.
3. **Matcher** — deterministically shortlists candidate quotes per requirement (generous, so nothing relevant is dropped).
4. **Verifier** — a deliberately **strict** agent that returns, for every requirement, `SUPPORTED | PARTIALLY_SUPPORTED | NOT_VERIFIED | NOT_FOUND` **plus the exact supporting quotes**, with anti-inflation rules baked into the prompt.

Every verdict is traceable to a resume sentence a human can check, and a recruiter review dashboard (`pages/7_📋_Candidate_Evaluation.py`) lets a human Confirm / Reject / Needs-review each verdict — human-in-the-loop, not autopilot.

Measured improvement

We compare the agent against the single-pass baseline on a **controlled suite of 10 hand-labelled cases** (see `evaluation/`), where the ground truth is the resume's *actual* support, not what a model claims. The cases are deliberately loaded with the traps the baseline gets wrong.

- **Reproducibility is guaranteed and verified:** the deterministic harness produces byte-identical results across runs and `PYTHONHASHSEED` values (three fresh runs verified).
- **Real Gemini numbers** require `GOOGLE_API_KEY`; the one-command run and the current measured table live in the changelog below.

Improvement Changelog

> Integrity rule: **we never invent numbers.** Every figure comes from an actual `python evaluation/run_evaluation.py` run.

Baseline

The pre-existing single-pass workflow (resume + JD → one holistic judgement), captured in `docs/baseline-record.md` before any challenge changes. Deterministic baseline on the 10-case

suite: **45.0%** evidence accuracy.

Experiment 1 — Structured Evidence Extraction

Added the EvidenceExtractor so matching is driven by quoted, structured claims instead of re-reading the whole resume. **Result: pending real run** (needs a Gemini key). **Decision: Kept** — extraction is the foundation of the pipeline.

Experiment 2 — Evidence Verification with strict prompts

Added the strict Verifier + deterministic Matcher to stop verdict inflation (skills-list ≠ SUPPORTED, "cloud" ≠ AWS, "working knowledge of GraphQL" ≠ GraphQL). **Result: pending real run**. **Decision: Kept** — strictness is the core defence against false positives.

Experiment 3 — Human review loop

Wrapped agent output in a recruiter review dashboard with Confirm / Reject / Needs-review and JSON export. **Result:** workflow usability (no accuracy metric applies). **Decision: Kept.**

Current measured state (10-case suite, deterministic smoke test)

From `python evaluation/run_evaluation.py --heuristic` — both systems run the same deterministic evaluator here, so Δ = 0 is expected by construction. This proves wiring + reproducibility, not the real comparison:

Case	Baseline	Agent	Δ
case_01_backend_engineer	20.0%	20.0%	+0.0pp
case_02_data_analyst	60.0%	60.0%	+0.0pp
case_03_frontend_engineer	60.0%	60.0%	+0.0pp
case_04_ml_engineer	50.0%	50.0%	+0.0pp
case_05_devops_engineer	75.0%	75.0%	+0.0pp
case_06_fullstack_engineer	20.0%	20.0%	+0.0pp
case_07_product_manager	25.0%	25.0%	+0.0pp
case_08_mobile_developer	25.0%	25.0%	+0.0pp
case_09_data_engineer	40.0%	40.0%	+0.0pp

Case	Baseline	Agent	Δ
case_10_qa_automation_engineer	75.0%	75.0%	+0.0pp
Aggregate (10 cases)	**45.0%**	**45.0%**	**+0.0pp**

Real agent comparison (headline measurement)

Run both systems on the same 10 cases with Gemini:

```
GOOGLE_API_KEY="AIza..." python evaluation/run_evaluation.py
```

This writes `evaluation/baseline_results.json`, `evaluation/agent_results.json`, `evaluation/comparison.md`. **Aggregate: pending real run.** Full details, including the hot take on the failure mode this project exists to fix, are in `docs/improvement-changelog.md`.

Reproduction

See `REPRODUCTION.md` for a clean-environment guide: exact commands for the app, the baseline, the agent, and the evaluation harness; required data; expected output; tool versions; approximate runtime and cost.

The agent at work (live)

From the repo root:

```
pip install -r requirements.txt
streamlit run app.py
```

Open the 📋 **Candidate Evaluation** page, paste a resume + job description, and run it. If `GOOGLE_API_KEY` is absent, the app degrades to a deterministic evaluator instead of crashing — so it always works, and the same code path that serves the UI is what the evaluation harness measures.

Repo map (Micro1 deliverables)

Path	Purpose
`ai/`	Agent package: models, resilient Gemini client, prompts, agents (RequirementExtractor, EvidenceExtractor, Matcher, Verifier), `pipeline.py`
`pages/7_📋_Candidate_Evaluation.py`	Recruiter review dashboard for the agent
`evaluation/`	Controlled baseline-vs-agent harness + **10 labelled cases**
`agent-traces/`	Agent trajectory log (instructions → tool calls → responses → feedback)
`docs/`	`baseline-record.md`, `improvement-changelog.md`, `micro1-git-workflow.md`
`REPRODUCTION.md`	Clean-environment reproduction guide
`notebooks/`	GPU-prep notebooks (ROCm / PyTorch / Hugging Face)

The pre-existing ResumePilot resume-optimization app (`app.py`, `pages/4–6`) remains intact and runnable — it is the challenge baseline.

Try it live

- **App:** https://resumepilot-ai-vngmvb9m6rdgszr7bthtbk.streamlit.app/
- **Repo:** https://github.com/abduaali132012-hash/ResumePilot-AI